

From Drift to Coherence

Implementation Summary: Martingale Property Assessment & PPR

1. Methods to Assess the Martingale Property

The paper tests whether LLMs satisfy the **martingale property**: that the model's predictive belief θ_n does not drift systematically when conditioned on its own past answers. Formally: $E[\theta_{n+k} | A_{1:n}] = \theta_n$.

1.1 Prompted Predictive Resampling (PPR)

The core protocol. The LLM is asked to answer the same question N times in a single generation call, producing a list of answer letters separated by newlines (e.g. A\nB\nA\nC\n. . .). The empirical frequency of this list is taken as the estimated predictive distribution θ_N . Multiple *independent* rollout trajectories (J paths) are averaged to estimate the martingale posterior mean q^* .

- **gen_len**: number of answers generated per call (paper uses 100 for local models).
- **J trajectories**: independent rollouts averaged for the posterior mean estimate.
- Each trajectory starts from the same prompt; conditioning on the full answer history creates the feedback loop that drives self-stabilisation.

1.2 Answer Seeding

Before running PPR, generate m i.i.d. seed answers from the model using a simple direct-query prompt (no resampling instruction). Prepend these seeds to the PPR prompt context. Seeding approximates the model's mean-field predictive uncertainty early on, accelerating stabilisation by reducing the effective burn-in period.

- **NoSEED**: no seeds prepended.
- **SEED1**: 1 seed answer prepended.
- **SEED5**: 5 seed answers prepended.

1.3 Drift Diagnostics

EMD (Expected Martingale Drift)

Average L1 distance between the 1-step prediction $p^{(1)}$ and the k -step-ahead prediction $p^{(k)}$, averaged over all k and all questions. Summarises overall martingale violation across the full trajectory.

$$EMD = (1/K) * \sum_k E_{\{Q, A_{1:n}\}} [d(p^{(k)}, p^{(1)})]$$

EMD_{k(b)} — Convergence Speed

Same k -step drift measured at burn-in offset b along the trajectory. Isolates early drift and shows how quickly the process stabilises.

$$EMD_{k(b)} = E_{\{Q, A_{1:n}\}} [d(p^{(k)}_{n+b}, p^{(1)}_{n+b})]$$

Distance metric d is the **L1 norm** (total variation) in the main paper; KL divergence is also reported in Appendix E.

2. Prompt Templates (Section D)

2.1 PPR Prompt — LLaMA-3.1-8B Style

Two-step generation rule inside a single chat completion call:

- **STEP 1:** Assign probability weights to each answer choice based on confidence.
- **STEP 2:** Output exactly *gen_len* lines, each a single letter sampled i.i.d. from that distribution, separated by `\n`.
- Format examples included in the prompt (e.g. `C\nA\nC\nA\n. . .`) serve as *structural* guidance only, not semantic demonstrations.
- Spaces, punctuation, and extra newlines are explicitly forbidden.

2.2 PPR Prompt — OLMo-3-7B Style

"Synthetic Data Generation Expert" framing with stronger format enforcement:

- Explicitly forbids cyclic patterns (ABAB, ABBABB, AABB, etc.).
- Instructs the model to restart internally if a pattern emerges.
- Omits the structural examples — relies on the format description alone.

2.3 Direct-Query (Seeding) Prompt

Intentionally simple, shared across all models and datasets. Used only to generate the seed answers before PPR:

```
Below is a multiple choice question.  
Return only one letter among {A, B, C, ...} which corresponds  
to your answer, without any spaces or newlines.  
Follow the below format:  
"Answer: [Write your answer choice letter here]"
```

3. Gap: Paper vs. Existing Codebase

The existing `run_martingale_check()` in `martingale_helpers.py` implements a **different** protocol from the paper's PPR:

Aspect	Current Code	Paper's PPR
Calls per step	1 call → full distribution	1 call → <i>gen_len</i> letters
Distribution estimate	Exact logprob or <code>n_api_samples</code> votes	Empirical frequency of list
History injection	"Your previous answers were: A, C"	Entire letter sequence prepended
Trajectories	1 path per question	J independent paths, averaged
Seeding	Not implemented	m i.i.d. seeds prepended
Drift metric	TV vs <code>p_0</code>	L1 between $p^{(k)}$ and $p^{(1)}$

4. Required Changes

4.1 New Prompt Templates

Add to `asym_duos/utils/system_prompt.py`:

- `ppr_prompt_llama(choice_str, num_choices, gen_len)` — STEP 1/STEP 2 style with structural examples.

- `ppr_prompt_olmo(choice_str, gen_len)` — Synthetic data expert style with anti-pattern enforcement.
- `direct_query_seed_prompt(num_choices)` — Simple single-answer prompt for seed generation.

4.2 PPR Answer Parser

Add to `asym_duos/utils/martingale_helpers.py`:

- Function that takes a raw model output string (e.g. "A\nB\nC\n...").
- Splits on `\n`, filters to valid label chars only.
- Returns an empirical frequency vector of shape `(n_classes,)` — this is $\theta_{\mathbf{N}}$.

4.3 New PPR Provider for API Models

Add `_build_ppr_provider()` in `asym_duos/utils/model_calls.py`:

- Takes `gen_len` and prompt style choice.
- Calls the API with `max_tokens` large enough for `gen_len` lines.
- Parses the response into an empirical frequency vector via the parser from 4.2.
- Runs J independent rollouts and averages them $\rightarrow q^*(x_Q)$.

4.4 New `run_ppr_check()` Function

Add to `asym_duos/utils/martingale_helpers.py` (separate from existing `run_martingale_check()`):

- For each question and each step n : runs J independent PPR rollouts $\rightarrow$ estimates $\theta_{\mathbf{N}}$.
- With seeding ($m > 0$): first generates m seed answers via direct-query calls, prepends them to the PPR prompt.
- Records the trajectory of θ_n across steps to compute EMD and $\text{EMD}_k(b)$.

4.5 New EMD Metrics

Add to `compute_martingale_metrics()` or a new function:

- **EMD**: compare $p^{\wedge}(1)$ vs $p^{\wedge}(k)$ for k in $\{2..K\}$, averaged over all n and all k .
- **EMD_k(b)**: fix burn-in offset b , compute k -step drift at position b along the trajectory. Produces the convergence-speed plots (Figure 2b in the paper).
- Both L1 norm and KL divergence variants should be supported (paper uses both).

4.6 New Config Parameters

Add a `ppr` sub-section to configs (or extend `api_model`):

- `gen_len`: answers generated per PPR call (paper: 100 for local, fewer for APIs).
- `n_trajectories (J)`: independent rollouts to average.
- `n_seeds (m)`: seed answers to prepend (0/1/5).
- `prompt_style`: "llama" or "olmo".

4.7 New Mode Flag in `martingale_property.py`

Add `--mode` argument to `tools/martingale_property.py`:

- `feedback` (current behaviour): iterative "Your previous answers were..." prompt injection.
- `ppr` (new): single-call list generation per trajectory step, using the new PPR provider.

Note: The most critical distinction: the current code measures how the model's distribution shifts when it reads back its past answers one-by-one. The paper's PPR measures how the model's empirical frequency over a long self-generated list evolves across independently seeded rollouts. These are two different experiments producing comparable but not identical drift measurements.

5. End-to-End PPR Implementation Flow

1. Load dataset & model/API provider	Same as current main() in martingale_property.py.
2. Select N questions	Same as current: rng.choice without replacement.
3. [Optional] Generate m seed answers per question	Call direct_query_seed_prompt → API → parse single letter. Repeat m times independently. Collect seed list $S_{\{1:m\}}$.
4. Build initial PPR prompt	Prepend instruction I (ppr_prompt_llama or ppr_prompt_olmo) to question Q. If $m > 0$, also append $S_{\{1:m\}}$ to the prompt.
5. Run J independent PPR rollouts per question	For each rollout $j=1..J$: call API with the PPR prompt, parse gen_len letter outputs → empirical frequency $\theta_{\blacksquare_N}(j)$.
6. Compute posterior mean q^*	Average the J frequency vectors: $q^* = (1/J) \sum_j \theta_{\blacksquare_N}(j)$.
7. Advance the resampling chain (step $n \rightarrow n+1$)	Sample one answer from q^* , append to the letter sequence in the prompt, repeat steps 4–6 for the next step.
8. Record distribution trajectory	Store q^*_n for $n = 0, 1, \dots, K$. Shape: $(N, K+1, n_classes)$.
9. Compute EMD and EMD_k(b)	Compare $p^{(k)}$ vs $p^{(1)}$ at each horizon k and burn-in offset b.
10. Save results	Same npz format as current save_martingale_results(), add EMD arrays.